Utilizing Reinforcement Learning with Path Planning Algorithms to Create More Efficient Paths of Travel

Andrew Meyer
Robotics Engineering
Worcester Polytechnic Institute
Worcester, Massachusetts
ameyer2@wpi.edu

Everett Wenzlaff
Robotics Engineering
Worcester Polytechnic Institute
Worcester, Massachusetts
ecwenzlaff@wpi.edu

Sushant Raj
Robotics Engineering
Worcester Polytechnic Institute
Worcester, Massachusetts
skraj@wpi.edu

***Abstract*—In this report, the implementation of a reinforcement learning algorithm in conjunction with multiple basic path planning algorithms is investigated to determine if reinforcement learning can efficiently optimize a simulated TurtleBot in a 2D environment based on the manipulation of several key parameters with the performance measured against path length, smoothness, travel time, velocity, and clearance. The simulations under study were conducted with ROS2 in a Gymnasium environment using open-source Turtlebot code and a Nav2 stack. The reinforcement learning algorithm was designed to utilize MPPI and DWB path planners and as a result, optimize them to perform better than their current offline counterparts.**

I. Introduction

A. Current state of the Art

In the industrial environment of Autonomous Mobil Robots (AMRs), path planners are generally deployed en-mass and are not tailored to the customers or environment needs in which they are deployed. This results in good, but not excellent performance, and while easier to maintain on the backend, can limit customer output on the front end. These general deployments often cast a wide net and aim to operate on the "safe side", whether that be wide, nonadjustable clearance, paths around high traffic areas, or creating lanes that avoid foot traffic altogether. In reality, each customer has different needs, and outside of required safety standards, may have certain workplace standards that could offer better performance if utilized properly to tune the AMR deployment. It is obvious that hand tuning any number of parameters for each individual deployment site is unreasonable and the time spent on it would far outweigh any long-term benefits. Additionally, the deployment time for standard AMRs can be several weeks, meaning there is a potential for downtime when looking at brownfield sites. Implementing a reinforcement learning (RL) algorithm paired with machine vision such as LiDAR, could reduce that deployment time from weeks to days, as all the training is done on the back end and there is no need to manually set path layouts.

B. Project Objectives

To reduce the amount of manual input needed to tune path planner parameters, an RL algorithm is deployed into the simulation for a period of training and the following parameters are adjusted to optimize the Turtlebot travel across the simulated warehouse: Clearance, Accuracy, Efficiency, and Smoothness. These four parameters have been previously proven to be statistically significant modifies for both path planners under study [1] [2]. In this context, a dense reward function is custom-built to help direct the RL algorithm through each episode so that when it is deployed, it can be more lightweight in terms of computational load.

C. Background

While modern RL algorithms are a fairly new technology, AMRs and their predecessors, Automated Guided Vehicles (AGVs), are not and have been in use for over 70 years in various forms. Similarly, path planners such as Model Predictive Path Integral (MPPI) and Dynamic Window Approach-Based (DWB) are 10-15 years old. As a result, there is a wide breadth of research that has been conducted to both verify plausiblity as well as serve as a reasonable set of starting points.

D. Path planning

Elbouhy et. al designed a study that consisted of three different path planners that were tested in environments with static and dynamic obstacles and the results were then evaluated to compare algorithms. The study analyzed DWB, MPPI, and RPP (Improved Pure Pursuit) planners and used the following criteria to determine path quality: time, distance, smoothness, minimum clearance, runs completed, and number of recoveries. This study showed that MPPI was the fastest but least smooth, DWB operated with the least clearance, and RPP took the most time and traveled the longest distance [1]. This demonstrates that different planners have different strengths and weaknesses which ultimately led to the integration of both MPPI and DWB into the RL-based simulation built in this project.

Li et. al studied the Dynamic Window Approach (DWA) algorithm and adjusted evaluation parameters in an effort to optimize the path planning. The evaluation function utilized in this paper looked at the heading, the distance between the trajectory and nearest obstacle, and the speed of the robot, along with weighted modifiers of each. A simple environment was setup and the tuning modifiers were adjusted for optimization [2]. In the context of this project, the interest lies in the tuning equation itself; and Li's tuning equation provided a confident proof-of-concept that path planning algorithms can be tuned and optimized on a limited number of parameters and the results are statistically significant and positive.

E. Reinforcement Learning

Qu et. al. presented a method of training an RL policy offline to accelerate the convergence for the online control of MPPI in an unmanned aerial vehicle (UAV). The method they proposed involved utilizing the Distributional Soft Actor-Critic (DSAC) algorithm to have an RL agent train a pair of neural networks to model distributions for the control costs and rollout guiding policy used directly within the online MPPI control. This improved MPPI by providing better "first guesses" than standard MPPI. The results of their experiments showed that their method converged significantly faster, produced smoother trajectories, and provided higher success rate in reaching the goal than standard MPPI [3]. The guiding principle of this paper translates directly to this project to enhance the strengths of existing local planners instead of replacing them by leveraging the strengths of both RL and traditional path planning algorithms to create a more efficient and adaptable path planning solution for mobile robots in complex environments.

Ting et. al. presented a modified Proximal Policy Optimization (PPO) in conjunction with a deep reinforcement learning (DRL) algorithm in order to reduce the time needed to move an agent through an environment. To reduce the number of episodes required and thus the calculation time for each move, they proposed a distance-based weighted dense reward function. The reward function they proposed utilized the euclidean distance to the goal to provide a higher value reward for better steps [4]. Since this reward function is not strictly linked to PPO parameters, the concept can easily translate into both MPPI and DWB that are utilized for the training in this project.

Debnath et al. propose a modular navigation framework for navigation that combines a classical global planner with a DRL based local planner. The global planner generates waypoints and assigns Gaussian soft costs around detected pedestrians to anticipate their future positions, while the trained local planner dynamically follows these waypoints while avoiding imminent collisions in real time. The reward function used to train the local planner includes eight terms covering goal reaching, collision penalties, waypoint tracking progress, pedestrian proximity avoidance, and a small per-timestep penalty to encourage efficiency. Through experiments, their hybrid system outperformed both a traditional DWA planner and a standalone RL baseline, with the RL-augmented system achieving a notably lower collision rate [9]. This paper is directly relevant to the dynamic parameter tuning approach adopted in this project where the RL agent observes local state and adjusts planner parameters throughout the navigation process rather than making a single configuration decision upfront.

Rana and Kaveendran implemented a PPO-based DRL agent to train a TurtleBot3 navigation policy from scratch within a ROS2 environment. The agent observes the robot's surroundings through a LiDAR scan and directly outputs normalized linear and angular velocity commands at each timestep, making it the sole navigation controller rather than an augmentation of an existing planner. Training was conducted using a two-term reward function including a small positive reward for forward progress and a large negative penalty for collisions. The trained policy resulted in better performance, and behavior improvements such as slowing near tight spaces [10]. This paper is particularly relevant as a validation of a similar implementation being used in this project: ROS2, Gymnasium, PPO, and a TurtleBot platform. The key distinction is that their RL agent replaces the planner entirely by outputting direct velocity commands, whereas this project's agent acts as a meta-controller. Their training time was approximately four hours for a significantly harder problem (the agent must learn full navigation from raw sensor data), providing reassurance that the team's proposed approach is viable.

F. Other Noteworthy Resources

In addition to the papers described in detail above, there were several other papers that were reviewed which provided supplemental information for shaping the project. [5] and [6] provided metrics and insights on planner path evaluation which was utilized as inspiration for the evaluation metrics implemented in this project. [7] and [8] demonstrated more examples which utilized RL within some sort of path planning framework for a mobile robot (including both direct and indirect RL control), again reinforcing the possibility of the concept explored in this project.

While there are several papers which demonstrate the use of RL in path planning, no papers were identified which explicitly utilized RL to configure the parameters for a given local path planning algorithm. This is a critical gap in the literature and study of this field, as it is well established that the performance of local path planners can be highly sensitive to their parameter settings, and that optimal settings can vary significantly across different environments and use cases. Meanwhile, some of the examples utilizing direct RL control are not very extensible as these methods tend to only train the RL agent to operate in a specific environment; and re-training for more complex environments can be very costly. By leveraging RL to automatically tune these parameters, this project theoretically fills this gap and demonstrates a novel approach to optimizing path planning performance in a way that is adaptable to the specific needs of different deployment sites.

II. Materials and Methods

A. Simulation setup

In order to reduce the scope and complexity of the required programming of this project, an RL agent was implemented within ROS2. Using ROS2 prevents needing to develop a simulation environment and navigation stack from scratch; which allows for more dedicated effort to developing the RL infrastructure and evaluating performance. Additionally, the simulation environment is constrained to a 2D grid with a TurtleBot robot model in order to simplify the state space and holonomic constraints for the path planners and RL infrastructure, again to avoid adding unnecessary complexity.

The environment was designed to be cross-platform and reproducible, leveraging Docker to encapsulate all dependencies and ensure consistent behavior across the team's Mac, Windows, and Linux development machines. The environment is structured as a two-layer Docker image: a base capstone_ros_layer image that installs all system-level dependencies (ROS2, Nav2, TurtleBot4 description packages, and visualization tools), and a top-level rbe_capstone image that layers in the project's ROS2 workspace. Separating dependency installation from application code allows the more expensive base layer to be rebuilt infrequently while code iteration remains fast.

The simulation runs on ROS2 Jazzy with the CycloneDDS middleware layer for inter-node communication. The robot model used is the TurtleBot4, whose URDF is loaded via `xacro` and published to the `/robot_description` topic by the `robot_state_publisher` node. Navigation is performed in a 2D occupancy grid environment using the Nav2 stack, which includes `nav2_map_server` for loading prebuilt maps and `nav2_lifecycle_manager` for managing the lifecycle of navigation nodes. Maps are stored as paired `.pgm` and `.yaml` files and are selectable at launch time, enabling the team to quickly swap between different warehouse environments as the project progresses.

Visualization is handled by RViz2, which renders the robot model, occupancy map, and planned paths. For the purpose of this project, RViz2 is chosen over Gazebo for visualization because it is more lightweight and the scope of this project does not require a full physics simulation. Gazebo will be consistently slower in launching and running episodes, which is not ideal for iterative tests and tuning of the RL algorithm. RViz2 provides sufficient visualization capabilities for the 2D occupancy grid and planned paths while allowing for faster iteration cycles. To further speed up iterative development, launch files are configured to bring up all required nodes—the robot state publisher, static TF broadcasters, map server, lifecycle manager, and RViz2—making it straightforward to start up the full environment with a single command.

B. RL Architecture

The RL component of the project is a trainable RL agent that tunes planner parameters directly through the Nav2 stack. Independent from the specific path planning algorithm, two design options initially were explored for RL architecture of the states, a one-time parameter tune, or a dynamic parameter tuning. It was decided that the implementation and expected results of a dynamic RL application would be favorable and feasible within the scope of this project. With this approach, the RL agent would be responsible for dynamically tuning the local planner's parameters throughout the navigation process where the state would includes the robot's current pose, velocity, distance from the goal, and information derived from on-board sensor data (e.g. downsampled LiDAR returns or costmap). The state vector is augmented to include prior poses and future waypoints from the global planner to provide more context for decision making. RL actions include presets for the local planner parameters (e.g. increase clearance by 0.1m, decrease path smoothness by 0.05, etc.).Finally, the reward function resembles a standard direct function with a large positive reward for reaching the goal, a large negative reward for collisions, and small negative rewards for all other movements to encourage finding the shortest path to the goal.

1) Training Architecture:

2) Reward Function:

3) Training and Evaluation Tools:

4) Performance Evaluation: In order to assess the impacts of incorporating RL within the local path planning process, a set of path quality metrics was used to evaluate the performance of the RL-augmented planner against a baseline planner with standard parameters across a variety of scenarios (e.g. different combinations of maps, start poses and goal poses). The path quality evaluation metrics will be based on the following criteria:

- Clearance from obstacles
- Accuracy (success rate in reaching the goal, amount of deviation from global path plan, etc.)
- Efficiency (total time and distance to goal)
- Smoothness (heading angle variance, velocity variance, etc.)

C. Code Architecture

—PRESENT CODE FLOW / BREAKDOWN OF KEY FILES—

1) Planner control and Episode Runner: These two ROS2 nodes (planner_controller.py and episode_runner.py) work in concert with each other and act as the backbone for the episodic training that is needed for the RL agent as well as building a basic baseline to compare it against. The planner node is able to dynamically switch between the DWB and MPPI planners and then loads and maps the correct parameters (linear and angular acceleration and velocity, as well as required distances) to pass into the episode runner. It also acts as a go-between to stored and communicate parameters as they change throughout the RL training and pushes them out to the episode runner. —-DIAGRAM—-

The episode_runner node, as the name implise, runs a loop of episodes for both the untrained (benchmark) and trained (RL mode) path planners. It sets up the simulation environment including the map and Turtlebot, and metrics for data

collection. During the episodes and once the Nav2 finishes, it fetches the designed performance metrics for analysis and comparison. After each timestep in the simulation, this node works with the planner to capture the required data for the rest of the RL architecture. —-DIAGRAM—-

III. Experimentation

A. Standard Path Planners

To generate an accurate baseline and determine the effectiveness of the RL agent, —-XX—- simulations were run with the off-the-shelf MPPI planner and DWB planner. Each simulation was evaluated for clearance, accuracy, efficiency, and smoothness and the results for both can be seen below. The MPPI had an average ——Placeholder for raw data —— ——TABLE / GRAPHICS ———.

B. RL-Trained Planners

For the RL agent, it was first trained —-XX—- and then deployed in the same environments as the above planners. The path was then evaluated against the same four qualifies and comparisons were drawn. ——Placeholder for data —— —— TABLE / GRAPHICS ———. A detailed analysis of the data can be seen below in the Results section.

IV. Results

After conduction all ——NUMBER—- of experiments with both standard and RL-driven planners, it was made clear that ——BIG CONCLUSIVE RESULT——

——GRAPHIC OVERLAYS OF STANDARD / RL PERFORMANCE FOR CLEARANCE, ACCURACY, EFFICIENCY, AND SMOOTHNESS——

Figures 1 and 2 below showcase the current state of the simulation environment as described above.

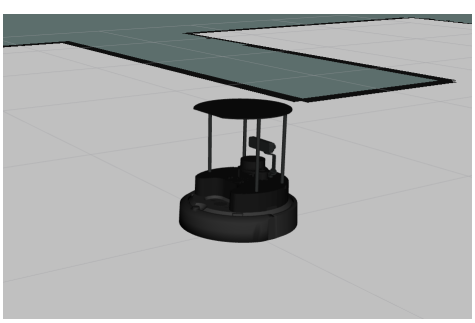
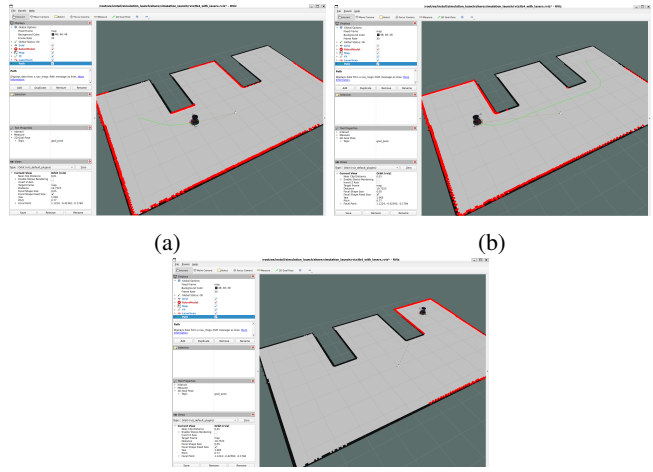


Fig. 1: TurtleBot4 model loaded in RViz2 with a simple occupancy grid map environment for demonstration purposes.



(a) (b) (c)

Fig. 2: TurtleBot4 model scanning and moving through the environment with goal poses set directly through RViz2. In 2a, the robot moves along the generated path (in green) from its initial pose to the first goal pose (in the upper left corner). Then in 2b, a new goal pose was commanded, resulting in a new path (to the upper right corner). Finally, in 2c, the robot reaches the final goal pose. All this navigation was done through manual control; and this sort of process will need to automated in order for the RL agent to be trained. Additionally, as the robot travels from 2a to 2c, it's clear from the LiDAR model scans (in red) that different parts of the environment become occluded. This is an important aspect of the environment that the RL agent will need to learn to account for when tuning the planner parameters.

V. Discussion

A. Implications of Results

B. Future Work

REFERENCES

[1] S. M. Elbouhy, A. Adamanov, P. M. Braun and H. Wilhelm Rose, ”Comparative Analysis of Local Trajectory Planning Algorithms in ROS2,” 2025 IEEE International Conference on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAR), Palermo, Italy, 2025, pp. 1-6, doi: 10.1109/SIMPAR62925.2025.10979015.

[2] H. Li, W. Zhang, Z. Dong, J. Yang and J. Lu, ”Local Path Planning Algorithm of Mobile Robot by Modified Optimized Parameters Method,” 2021 IEEE 5th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC), Chongqing, China, 2021, pp. 1014-1018, doi: 10.1109/IAEAC50856.2021.9391102.

[3] Y. Qu et al., ”RL-Driven MPPI: Accelerating Online Control Laws Calculation With Offline Policy,” in IEEE Transactions on Intelligent Vehicles, vol. 9, no. 2, pp. 3605-3616, Feb. 2024, doi: 10.1109/TIV.2023.3348134.

[4] L. Ting, H. B. Y. Ali and M. K. Chamran, ”Accelerating Deep Reinforcement Learning for Mobile Robot Path Planning with Curriculum Learning and Reward Shaping,” 2025 IEEE 9th International Conference on Software Engineering and Computer Systems (ICSECS), Pekan, Pahang, Malaysia, 2025, pp. 343-347, doi: 10.1109/ICSECS65227.2025.11279207.

[5] F. Pimentel and P. Aquino, “Performance Evaluation of ROS Local Trajectory Planning Algorithms to Social Navigation,” Oct. 2019, doi: https://doi.org/10.1109/lars-sbr-wre48964.2019.00035.

[6] Isira Naotunna and Theeraphong Wongratanaphisan, “Comparison of ROS Local Planners with Differential Drive Heavy Robotic System,” vol. 4, pp. 1-6, Dec. 2020, doi: https://doi.org/10.1109/icamechs49982.2020.9310123.

[7] C.-C. Wong, K.-D. Weng, and B.-Y. Yu, “Multi-Robot Navigation System Design Based on Proximal Policy Optimization Algorithm,” Information, vol. 15, no. 9, p. 518, Aug. 2024, doi: https://doi.org/10.3390/info15090518.

[8] F. Almeida, G. Leão, and A. Sousa, “An Educational Kit for Simulated Robot Learning in ROS 2,” Robot 2023: Sixth Iberian Robotics Conference, vol. 2, pp. 513-525, doi: https://doi.org/10.1007/978-3-031-59167-9_4.

[9] A. Debnath, G. J. Stein, and J. Košecká, ”A Hybrid Approach to Indoor Social Navigation: Integrating Reactive Local Planning and Proactive Global Planning,” *2025 IEEE International Conference on Robotics and Automation (ICRA)*, Atlanta, USA, May 2025, pp. 10432–10438, doi: https://doi.org/10.1109/ICRA55743.2025.11127475.

[10] A. Rana and B. Kaveendran, ”Deep Reinforcement Learning with PPO for Autonomous Mobile Robot Navigation Using ROS 2 Framework,” *International Journal for Research in Applied Science and Engineering Technology (IJRASET)*, vol. 13, no. VII, Jul. 2025, doi: https://doi.org/10.22214/ijraset.2025.73330.

APPENDIX

A. Source Code

All the code for this project can be found in a Git repository here: [https://github.com/SushiRaj99/rbe-capstone] or seen below.